

Secure Communication System

CA One (B9IS129) — Complete Feature Breakdown for Group Members
Mubashir (Security & Crypto) · Hamza (Server & Deployment) · Kiran (Client & Auth UI)

1. Purpose of this document

This PDF summarises everything implemented in the project so far: original CA_ONE security requirements plus later enhancements (MongoDB Atlas, JWT, OTP, chat UI, etc.). Use it to align ownership, prepare the report/viva, and avoid duplicated work.

2. Group ownership (agreed split)

Member	Primary responsibility	Main folders / areas
Mubashir	Security & encryption core, threat model, crypto docs/tests	crypto_core/, docs/security_*, tests/
Hamza	Backend relay server, MongoDB, JWT APIs, Socket.IO, deployment	server/, .env.example, deployment config
Kiran	Client UI, browser crypto wiring, auth screens, chat UX	client/, templates, static JS/CSS

Note: Some features cross boundaries (e.g. OTP needs server + client). The tables below mark Primary owner and Supporting owner.

3. Original CA_ONE core features (must-have)

Feature	What it does	Primary	Support
ECDH key exchange (no prior meeting)	Browsers derive a shared secret from public keys via the relay server.	Mubashir / Kiran	Hamza (store/fetch keys)
AES-256-GCM message encryption	Messages encrypted in browser; server only relays ciphertext.	Mubashir / Kiran	Hamza (relay)
Maliciously curious server model	Server cannot read plaintext; designed as untrusted for confidentiality.	Mubashir	All
Public key directory	POST /store_key + GET /get_key for peer public keys.	Hamza	Kiran
Real-time relay (Socket.IO)	send_message → receive_message between users.	Hamza	Kiran
Replay attack protection	Unique msg_id + 5-minute seen-ID window (ReplayGuard).	Hamza	Mubashir (threat analysis)
Key fingerprint UI	Out-of-band MITM check after session establish.	Kiran	Mubashir
Register / Login (baseline)	User accounts before chat.	Hamza / Kiran	—
crypto_core Python module	Reference ECDH/AES/fingerprint + pytest round-trip.	Mubashir	—
Architecture / design docs	README + docs/ design, architecture, security requirements.	All	Kiran (client docs)

4. Enhanced features added later (extensions)

These go beyond the minimum brief and strengthen security, usability, and deployability.

4.1 Backend / server / data (Hamza primary)

- **MongoDB Atlas integration** — database *Secure_Communication*; collections: users, otps, sessions, password_resets, messages, security_logs (+ indexes/TTL).
- **Environment secrets** — .env / .env.example; no hardcoded DB credentials or JWT secrets in source.
- **AuthManager on MongoDB** — register/login/profile/password reset backed by Atlas (replaced users.json for new accounts).
- **bcrypt password hashing** — replaces plain SHA-256; password complexity rules (8+, upper/lower/digit).
- **JWT access tokens + refresh token rotation** — short-lived access JWT; hashed refresh tokens in sessions collection.
- **Protected REST + Socket.IO** — Bearer JWT required for keys/profile/messages; socket identity from JWT (not spoofable username alone).
- **Email OTP service** — 6-digit OTP, hashed at rest, 5 min expiry, 60s resend, max 3 attempts, one-time use; Gmail SMTP support + EMAIL_DEV_MODE.
- **Login OTP (2FA step)** — after password OK, login completes only after email OTP verification.
- **Registration email verification OTP** — account must verify email before full login.
- **Forgot / reset password** — hashed reset tokens, expiry, email link to reset page.
- **Rate limiting** — flask-limiter ~5/min on auth endpoints → HTTP 429 + security log.
- **Security audit logs** — login/logout/fail/OTP/reset/rate-limit/replay events with IP/UA/time.
- **Message persistence (ciphertext only)** — history stored without plaintext; soft delete/edit ciphertext APIs.
- **Conversations API** — GET /conversations for sidebar chat list + online flags.
- **User lookup API** — GET /lookup/<username> distinguishes user_not_found vs key_missing (stops endless polling).
- **CORS lockdown + Talisman headers** — restricted origins; security headers (local HTTP-friendly).
- **Health check** — GET /health (Mongo ping).
- **Docs for setup/API** — docs/SETUP_AND_API.md.

4.2 Client / UI / auth UX (Kiran primary)

- **Modern auth page** — Login/Register tabs only; Forgot password as link; OTP as step screen.
- **Resend OTP cooldown UI** — button disabled 60 seconds with countdown.
- **Button loading locks** — disable while API in flight (login/register/OTP/forgot).
- **JWT session in localStorage** — access/refresh tokens; redirect to chat after login OTP.
- **WhatsApp-style chat layout** — left sidebar (profile + chat list), right conversation pane.
- **Conversation list + search + new chat** — previous chats, online dots, start chat by username.
- **Persistent browser ECDH/AES keys** — survive refresh so history can decrypt again.
- **Typing indicators, presence, read/delivery status** — Socket.IO events wired in UI.
- **Loaders & skeletons** — app connect, sidebar, messages, profile page skeletons.
- **Unregistered user handling** — lookup before open chat; clear error state; no infinite get_key loop.
- **Modern profile page** — live avatar preview, sections, validation hints, danger zone.
- **Avatar images** — direct image URL rendering with fallback initials.
- **Logout confirmation modal** — Cancel / Log out dialog before signing out.
- **Establish encryption control** — manual ECDH retry if peer key not ready.

4.3 Security / crypto (Mubashir primary)

- **Threat model documentation** — maliciously curious server; what server can/cannot see.
- **crypto_core algorithms** — ECDH, AES-GCM, fingerprint helpers + pytest.
- **Algorithm justification** — why GCM, why ECDH, why fingerprint MITM check.
- **Security requirements & design specs** — docs used for CA_ONE write-up.
- **Review of applied controls** — bcrypt, JWT, OTP, rate limit, replay, ciphertext storage (coordinate with Hamza).
- **Known limitations to document** — fingerprint is manual; browser uses P-256 Web Crypto (align docs if README still says X25519); no Signal-style ratcheting.

5. Main API surface (Hamza / Kiran integration)

Area	Endpoints / events	Auth
Auth	/register, /verify-otp, /resend-otp, /login, /refresh, /logout, /forgot-password, /reset-password	Mixed (JWT on logout)
Keys	/store_key, /get_key/<user>, /lookup/<user>	JWT
Profile	GET/PUT /profile, GET /profile/<user>, /change-password, DELETE /account	JWT
Chat data	/conversations, /messages/<peer>, message edit/delete/search	JWT
Socket.IO	connect(auth.token), join, send_message, receive_message, typing, presence, message_status, message_read	JWT on connect

6. Suggested ownership for remaining CA_ONE work

Task	Owner	Why
Update README/docs: P-256 vs X25519, bcrypt (not SHA-256), OTP/JWT	Mubashir + Hamza	Markers check consistency
Demo script (2 browsers): register→OTP→login OTP→chat→fingerprint→replay	All (Kiran leads UI demo)	Viva / marking demo
Cloud deployment (Render/Azure/Railway) + HTTPS notes	Hamza	NFR deployment
crypto_core tests green + short security test evidence	Mubashir	Security ownership
Client polish + screenshots for report	Kiran	UI ownership
Group report sections + AI assistance log links	All	Submission requirement

7. What is enough vs optional for marks

Already enough for the brief: E2E chat without prior key meeting, untrusted relay server, identity approach, replay protection, docs, team split.

Extensions already done (bonus strength): MongoDB, bcrypt, JWT, OTP (register+login), rate limiting, audit logs, ciphertext history, modern UI.

Do not prioritise for CA_ONE unless spare time: groups, file sharing, video, admin dashboards. They add little security marks compared to clean docs + demo.

8. Key project paths

- Project root: **C:\Users\Supreme_traders\Projects\Secure_Communication_System**
- Server run: **python -m server.server** (port 5000)
- Client run: **python client/app.py** (port 3000)
- Setup/API doc: **docs/SETUP_AND_API.md**
- This PDF: **docs/CA_ONE_Feature_Breakdown_Group.pdf**

Prepared for group sharing — Secure Communication System (CA One). Keep secrets in .env only; never commit MongoDB passwords or Gmail App Passwords to GitHub.